Lecture 17 · Part IV · Lecture notes

Text

Dataset: IMDb film reviews

How to use these notes

The primary text for this lecture is Géron, Chapter 14. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 A new kind of input **2**

1.1 The brief . . . 2

1.2 The anchor, before any model . . . 3

2 Derivation: softmax, cross-entropy and logits **3**

2.1 The loss, and its gradient . . . 3

3 Tokens, embeddings, and a classifier **4**

3.1 What is a token? . . . 4

3.2 From integers to vectors . . . 5

4 The swap **6**

5 The notebook **7**

6 Design exercise: a customer-support SaaS's automatic ticket routing **7**

7 Exercises **9**

1 A new kind of input

Sixteen lectures on numbers, pixels and ordered numbers. Today the input is free text: variable length, discrete, and with no natural numeric encoding at all. A review is a sequence of *symbols*, not of measurements, and there is no distance between two words that anything gave us.

Everything in Chapters 1–13 arrived as a table — a fixed set of features, in a fixed order. A review is none of those: variable length, no fixed positions, and an *open* vocabulary, because the next review can contain a word that has never been written before. So before any model there is a question no previous application had to answer: *what is the unit?*

1.1 The brief

A streaming service's feedback desk wants, for each new review, a single label — satisfied or not — so the unhappy ones can be routed to a human within the hour.

Their words	**What that specifies**
"tell us if it is a complaint"	binary classification
"from the text alone"	one variable-length string in, one label out
"within the hour"	*inference* cost matters; training cost does not
"we will read the ones you flag"	the errors are absorbed by a human

Four sentences, and every one is a design decision you would otherwise have made silently.

Review length has a long right tail, with a mode near a hundred words. Wherever you cut, you cut something — so the 192-token cut used below is a decision to make on purpose and record: it truncates 43.8% of the training reviews and keeps 66.7% of all their words.

What the model has to produce is string $\to \mathbb{R}^2$: one score per class. Between the two sit four stages, and only the first two are new — the last two are Lectures 8 and 16 unchanged.

Stage	**In**	**Out**
Tokenize	one string	$(192,)$ integers
Embed	$(B, 192)$	$(B, 192, 128)$
Recur	$(B, 192, 128)$	$(B, 128)$
Head	$(B, 128)$	$(B, 2)$

What is the shape here? is the question that catches most of what goes wrong in what follows.

1.2 The anchor, before any model

Lecture 3 counts the classes before choosing a metric. They are 50.0% to 50.0%, so accuracy is defensible *on this corpus* — the degenerate classifier scores 50% and has nowhere to hide. Real feedback is not balanced, and the first slide of the next application should ask again.

A bag of words weights each term by tf-idf = $\text{count}(t, d) \times \log \frac{N}{1+n_t}$, the second factor being what stops *the* and *movie* dominating the first. Note what is *not* in that formula: position. The bag has no order, so it cannot see that *not* changes the word after it. (Scikit-Learn uses the smoothed variant $\log \frac{1+N}{1+n_t} + 1$ and normalises each row to unit length.) The vectoriser is a fitted estimator: `fit_transform` on the training reviews, `transform` on the test reviews — different verbs, and the difference is where Lecture 18 ends.

Model	Features	Test accuracy
Always predict one class	—	50.0%
tf-idf, single words	44,798	88.5%
tf-idf, words and pairs	200,000	90.1%

90.1% from counting words. Adding adjacent pairs buys 1.6 points, and pairs are the cheapest way a bag can see *not good*. The brief's other requirement — "within the hour" — is one the metric cannot see: a bag of words scores the whole test set long before the hour is up.

2 Derivation: softmax, cross-entropy and logits

Logits are the output of the last linear layer before anything is done to it: K unbounded numbers per input, one per class, not summing to anything.

Softmax maps them to a probability vector, $\sigma(\mathbf{z})_k = e^{z_k} / \sum_j e^{z_j}$: every entry positive, summing to one, and *order-preserving* — which already tells you that `argmax` does not need it. A prediction never requires softmax.

2.1 The loss, and its gradient

Cross-entropy between a target distribution $\mathbf{y}$ and a prediction $\mathbf{p}$ is $H(\mathbf{y}, \mathbf{p}) = -\sum_k y_k \log p_k$. Against a one-hot target every term but one dies, and it collapses to $L = -\log p_c$: one number, and unbounded when the model is confidently wrong. That is the loss `nn.CrossEntropyLoss` computes, and it is given the logits, not the probabilities, because it applies the softmax itself.

The gradient with respect to the logits

Substituting softmax into the loss,

$$L = -\log \frac{e^{z_c}}{\sum_j e^{z_j}} = -z_c + \log \sum_j e^{z_j}.$$

The exponential of the true class has cancelled. Differentiating, the first term contributes $-y_k$ and the second is the log-sum-exp, whose derivative is softmax:

$$\boxed{\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}}$$

Prediction minus target. Checked against autograd on one batch, the algebra and PyTorch agree to 5.55×10^{-17}.

Failure condition — the softmax applied twice

Apply a softmax yourself before a loss that expects logits and nothing raises, the model still trains, and the gradient it trains on is the wrong one: `nn.CrossEntropyLoss` applies the softmax itself, so $\mathbf{p} - \mathbf{y}$ is the gradient only when the softmax is applied once.

3 Tokens, embeddings, and a classifier

3.1 What is a token?

The unit the model counts as one thing. *Characters* give a tiny vocabulary, sequences six times longer, and every fact about a word relearned from letters. *Words* are the obvious choice, and the one that breaks. *Subwords* are the compromise the field settled on.

This is a modelling decision, not a preprocessing detail: it fixes the vocabulary, the sequence length, and what the model can possibly represent.

Our word tokenizer is one regular expression, and three arguable decisions are already inside it: case is destroyed, so *Terrible* and *terrible* become one token; punctuation is destroyed, and so is the exclamation mark; and the corpus is HTML, so `<br />` has to be removed. *If you had not looked at the raw text you would have learned* `br` *as one of the commonest words in the corpus.*

Failure condition — what a word vocabulary costs

Even at the full 42,370-word vocabulary of the fit split — every word in the 5,000 reviews a model fits on — **61.3%** of the *distinct* words in the test set have never been seen, while only 2.3% of test tokens are.
The token rate looks survivable. The rate over distinct words never does, and those are the informative ones.

Subword tokenisation is a structural fix rather than a patch. Start from characters, which cover everything by construction, and repeatedly merge the commonest adjacent pair until the vocabulary reaches the size you asked for. Common words survive whole; rare words become pieces. *The vocabulary cannot miss*: an unseen word falls back to pieces, an unseen piece to

characters, and the character set is closed — so there is no input for which the tokenizer must emit `[UNK]`. Morphology survives too: *un* + *watch* + *able* keeps the negation that `[UNK]` destroyed.

3.2 From integers to vectors

Token 4271 is not four thousand of anything. The integer is a *name*, and arithmetic on names is meaningless. One-hot encoding would give a 20,000-dimensional vector per token and 192 of them per review.

An **embedding** is a learned table with one row per token: looking up row i is exactly multiplying the one-hot vector by that table, done without ever forming the one-hot vector. It is a *parameter*, not a preprocessing step — every one of its numbers has a gradient — and `padding_idx=0` pins row 0 at zero and keeps it there, because padding must not learn anything.

The classifier is thirteen lines, eleven of which are Lecture 8. Bidirectional because a review's verdict can be in the first sentence or the last; the final hidden state because it is one fixed-size summary of a variable-length input; GRU rather than LSTM because it has fewer gates and parameters, and on Lecture 16's forecast it beat the LSTM.

`pack_padded_sequence`, and what it prevents

It hands the recurrent layer only the real timesteps, so the state returned for a review of 40 words is the state after its 40th word — not after 152 zeros.
Correctness, because the summary is of the review rather than of the padding; and speed, because the layer stops early instead of stepping through the whole rectangle.
This matters because the version *without* it also runs, also trains, and also reports a plausible number. A shape test cannot see that; an *invariance* test can — pad the same review two different ways and require the logits not to move.

Setting	Value
Reviews used to fit	5,000
Reviews held out for validation	2,000
Reviews in the test set, untouched	25,000
Epochs, batch size, learning rate	2, 64, 0.001
Parameters	2,634,754

The budget is small on purpose, so that the notebook runs on a free CPU, and every number below is from that run. The training loop is Lecture 8's five lines, unchanged, with `nn.CrossEntropyLoss` given the logits.

Model	Test accuracy
Always one class	50.0%
tf-idf + logistic regression	90.1%
GRU, our tokenizer, trainable embeddings	63.0%

The recurrent network, with an embedding it learned itself, loses to counting words, by 27 points.

Why? The bag starts with 200,000 features that are already meaningful, fitted on all 25,000 training reviews — *terrible* is its own column. Our network starts with 20,000 rows of random

numbers and has to discover from 5,000 labelled reviews, in two passes, that *terrible* and *awful* are related, when most of its words are rare in 5,000 reviews.

The hypothesis — and it is only a hypothesis

The architecture is not the bottleneck. The embedding table is, and it is being asked to learn the English language from 5,000 sentiment labels.
Written as a hypothesis on purpose. What follows is an experiment that can refute it, and you should decide now what result would.

4 The swap

Replace the table, and nothing else: same GRU, same width, same head, same optimiser, same learning rate, same two epochs, same seed, same 5,000 reviews. Two changes, applied *one at a time*, because changing both and reporting the total is how you learn nothing from an experiment.

The first swap is DistilBERT's tokenizer alone: subword pieces, 30,522 rows instead of 20,000, random initial values exactly as before. The second pours in the embedding table that vocabulary already has, trained on far more text than this corpus, by someone else, for a different task. That table is 768 columns wide and our architecture is 128, so it is projected to 128 columns with PCA before it goes into `emb.weight`; widening the GRU instead would change the architecture, and then the comparison would measure two things at once. Then one more decision, which is one boolean: *frozen*, so that only the GRU and head learn (the risk is that the vectors do not suit the task), or *fine-tuned*, so that everything learns (the risk is that 5,000 labels overwrite them). Which is better is not decidable from the armchair, so run both.

Tokenizer	Embedding table starts as	Validation accuracy
our words	random	64.4%
subword	random	58.4%
subword	pretrained, frozen	50.5%
subword	pretrained, tuned	79.4%

The swap is read on the 2,000 validation reviews, at each run's best epoch, so the test set chooses nothing. The subword vocabulary alone is −6.0 points, from one seed each — not enough to call a ranking, but certainly not the gain the fix promised: solving the out-of-vocabulary problem, on its own, bought nothing.

The pretrained vectors, fine-tuned, are worth +15.0 points, and not one line of the model changed — well beyond the swing we refused to call a ranking one row above. The frozen row says the vectors alone did not do it: held fixed, after two epochs the model is still at chance. The pretrained table is a starting point that the labels then adapt. Everything ordinarily tuned in an afternoon (width, depth, head, loss, optimiser, epochs, batch size, learning rate, seed) is unchanged; what moved is the tokenizer, the initial values in one tensor, and whether that tensor is trainable.

Text checklist

1. Was the vocabulary built on the training split only?
2. Is the tokenizer the same object at fit time and at inference?
3. Are true lengths carried alongside the padded batch?
4. Does padding change the output? Assert that it does not.
5. What fraction of the test tokens is `[UNK]`? Count it.
6. How much of each review survives truncation? Count that too.

The first is the one that will cost you the most, and it is the one the next lecture ends on.

5 The notebook

What to change

Concatenate `h[0]` with itself instead of `h[1]`. The shapes and the parameter count are unchanged, nothing raises, and the backward direction is now dead weight. What happens to the accuracy?

6 Design exercise: a customer-support SaaS's automatic ticket routing

The lecture closes with a design exercise in the manner of an ML system-design interview, framed as the ML system behind a company's product.

The task. A customer-support SaaS sells the helpdesk other companies run their support on, and is shipping automatic routing. For each ticket — free text written by a customer's customer — the system outputs the team that should handle it, from that account's own teams — billing, product areas, security — so that it lands in the right queue without an agent reading it first. The customers measure the time to first response their service agreements promise; the SaaS measures the share of tickets triaged by hand, and renewals. Given: each account's past tickets, with the team that closed each, and its current list of teams. Thousands of accounts, with thousands to millions of tickets each; a handful to dozens of teams per account, two or three of which take most of the traffic and some a few tickets a week. The output feeds the queueing system, the team leads' dashboards, and the triage view for whatever the model will not route. One model per account.

1. **Frame it.** One string in, one score per team out: softmax over the account's teams, cross-entropy on the logits, the class being the team that resolved the ticket. Count the classes before choosing a metric: the largest team alone takes most of the traffic, so the anchors are always-the-largest-team and then a bag of words with logistic regression, measured before any network. *Pitfall:* accuracy. A router that sends everything to the largest team scores the base rate and looks fine, while the small teams — where a misroute costs most: a security

report in the billing queue past the promised response time — are invisible in the number. Report the confusion matrix and each team's recall beside the base-rate anchor, and ask the account what a misroute costs per team before choosing what to optimise.

2. **The label.** The first assignment is the account's old rule, or a tired agent's, so label with the team that *closed* the ticket, and only closed tickets; flag the tickets that bounced between teams so their share can be read separately. *Pitfall:* the ticket that belongs to two teams — a refund and a login failure — which one softmax forces into one answer. Agree the rule with the account before labelling, the team that must act first owns it, and apply the same rule when the label is read back. Every later number is a number about that label.

3. **Split by time.** Train on the tickets up to a cut, validate on the months after it, test on the latest months, the order the router will face in the account's helpdesk; choose everything on the validation tickets and read the test tickets once. *Pitfall:* a random split. An outage generates thousands of near-identical tickets in one afternoon, and a shuffle puts half in training and half in test: the score climbs, and the next outage, about a new product, is the first the deployed model has ever seen.

4. **Tokens.** Look at the raw text first — quoted replies, signatures, mail headers, pasted logs — or the commonest tokens will be *wrote* and *regards*. Subword pieces, so that the account's product names, error codes and misspellings become pieces rather than an unknown token. Cut each ticket at a fixed length and count what survives: the subject and the opening of the body, where the problem is usually stated. *Pitfall:* the vocabulary built on every ticket, the test months included; nothing raises, and the model has a row for a product name that did not exist when it was supposedly fitted. Build the vocabulary on the training tickets only, keep the tokenizer as one serialised object per account used at fit time and at inference, and count the unknown-token rate on later tickets.

5. **The model.** Tokens, embedding rows, a bidirectional GRU packed to the true lengths, a linear head with one logit per team. The embedding table starts pretrained and is fine-tuned, because the small teams have a few hundred labels each; change one thing at a time against the bag-of-words anchor. *Pitfall:* a table of random numbers learned from the tickets alone, which trains, and loses to counting words by a wide margin, because every rare product name is one row that one example cannot teach. If the pretrained table, tuned, does not beat the bag of words on validation, ship the bag of words: it is cheaper to serve for thousands of accounts, and nobody bought a neural network.

6. **Silent failures.** A softmax before the loss trains on the wrong gradient: logits into the loss. Truncation that removes the problem routes long tickets by their greeting: keep the subject whole and count what survives. *Pitfall:* the state after the padding. The version without packing also runs, also trains, and also reports a number, and the summary of a short ticket is the state after a hundred zeros. Carry the lengths, pack the batch, and assert that padding the same ticket two ways gives the same logits — an invariance test, in the release's test suite.

7. **Early stopping.** Score the validation tickets after every epoch: the curve rises, peaks and falls as the few labels of the small teams are memorised. Keep the checkpoint of the best validation epoch; the number of epochs is a result, not a setting. Then the confusion matrix per team on the test months, read once, with the error count beside the anchors; a small gap from one seed is not a ranking. *Pitfall:* the last epoch shipped because that is what the loop leaves in memory, or the epochs chosen by watching the test set, which then chooses nothing.

8. **In production.** Released behind a per-account switch, to accounts that opt in first: route when the top team's probability is high and send the rest to triage, whose decisions are the freshest labels; monitor per account and per team the volume routed, the share re-routed by hand and the unknown-token rate; retrain each account on a recent window at a fixed cadence, with the data team on call for accounts whose re-routes jump. *Pitfall:* a customer reorganises in the admin panel — one team splits, another is merged, a product is retired — after which the label set the model was trained on no longer exists and tickets land in a queue nobody reads until the customer's own customers complain. Version each account's label set, map every historical label through each reorganisation before retraining, and treat an agent's re-route as the correction it is.

9. **Ship it.** Closed tickets labelled by the closing team → split by date → raw text inspected, boilerplate stripped → subword tokenizer and vocabulary fitted on the account's training tickets → bag-of-words anchor → pretrained embeddings, a packed bidirectional GRU, logits into the loss → best validation epoch kept → confusion matrix on the test months → opt-in rollout, triage the rest, monitor re-routes → retrain.

7 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 18's deck and in the exercise book.

1. Swapping the embedding table moved validation accuracy from 64.4% (our words, random vectors) to 58.4% with the subword tokenizer alone, and to 79.4% with pretrained vectors, tuned. Why were the two changes applied one at a time, and what does the frozen run add? [5]

2. State the gradient of cross-entropy with respect to the logits for a one-hot target, and what each row of that gradient must sum to. [5]

3. A classifier's `forward` ends in a softmax, and its output is passed to `nn.CrossEntropyLoss`. State what happens in training, and the fix. [5]

4. A padded batch is fed to a GRU without `pack_padded_sequence`. State what the returned state summarises for a review shorter than the cut, and a test that detects it. [5]

5. Enlarging a word vocabulary drives the share of unseen distinct test words down, but even the full 42,370-word vocabulary leaves 61.3% of them unseen. Explain why a larger vocabulary is not the fix, and name what is. [4]

Summary

Softmax maps the logits to a probability vector. Cross-entropy against a one-hot target is $-\log p_c$, unbounded when the model is confidently wrong, and its gradient with respect to the logits is exactly $\mathbf{p} - \mathbf{y}$, agreeing with autograd to 5.55×10^{-17}. `nn.CrossEntropyLoss` applies the softmax itself, so it is given logits; a softmax applied before it trains without complaint on the wrong gradient.

Then the application, which went only part of the way the phrase “transfer learning” suggests. A word vocabulary leaves 61.3% of distinct test words unseen even at 42,370 entries, and subword tokenisation removes that failure entirely — and bought *nothing* on its own. Pretrained vectors, fine-tuned, were worth 15.0 validation points with 5,000 labels and two epochs. And the recurrent model trained from scratch lost to tf-idf and logistic regression, 63.0% against 90.1% on the test set. The closing design exercise built a customer-support SaaS’s automatic ticket routing in nine steps — the teams counted before the metric, the closing team as the label, a split by date, a vocabulary built on each account’s training tickets only, a pretrained table tuned against the bag-of-words anchor, packing asserted, the best validation epoch kept, an opt-in rollout, and a label set versioned when a customer reorganises its teams.